

Scooter Rental Management System

Student Name: Hamdan

Student ID: 202204298

University: Zayed University

Course: _____

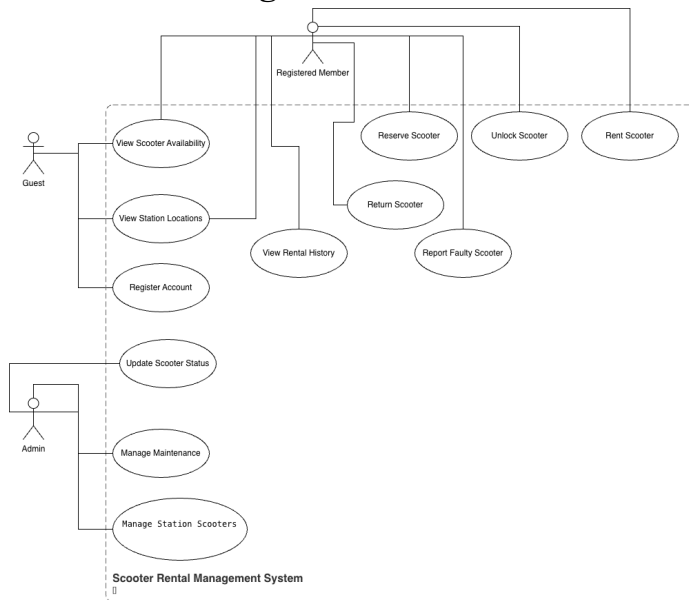
Instructor: _____

Date: _____

1. System Description

This project is about a scooter rental management system. The system helps users rent scooters in a simple and organized way. Users can see available scooters, reserve them, rent them, and return them after use. The system also manages stations, scooter status, and maintenance. The goal is to make scooter rental easy for users and easy to manage.

2. Use-Case Diagram



The use-case diagram shows how users interact with the system. The main actors are Guest, Registered Member, and Admin. Guests can view scooters and register. Members can rent and return scooters. Admin manages the system.

Each use-case is explained below:

- **View Scooter Availability**
The user checks which scooters are available.
- **View Station Locations**
The user sees where stations are located.
- **Register Account**
The guest creates an account to become a member.
- **Reserve Scooter**
The member books a scooter before using it.
- **Rent Scooter**
The member starts using a scooter.
- **Unlock Scooter**
The scooter is unlocked using the system.
- **Return Scooter**
The member returns the scooter after use.
- **View Rental History**
The member checks previous rentals.
- **Report Faulty Scooter**
The member reports a problem.
- **Manage Maintenance**
Admin updates scooter condition.
- **Manage Station Scooters**
Admin manages scooters at stations.

3. Class and Relationship Definitions

User

- user_id: int
- name: string

Member (inherits User)

- rentals: list

Methods:

- rent_scooter()
- reserve_scooter()

Scooter

- scooter_id: int
- battery_level: int
- status: string
- location: string
- scooter_type: string

Station

- station_id: int
- name: string
- capacity: int
- scooters: list

Methods:

- add_scooter()

Rental

- user: User
- scooter: Scooter
- start_time: int
- end_time: int
- cost: float

Methods:

- end_rental()

Reservation

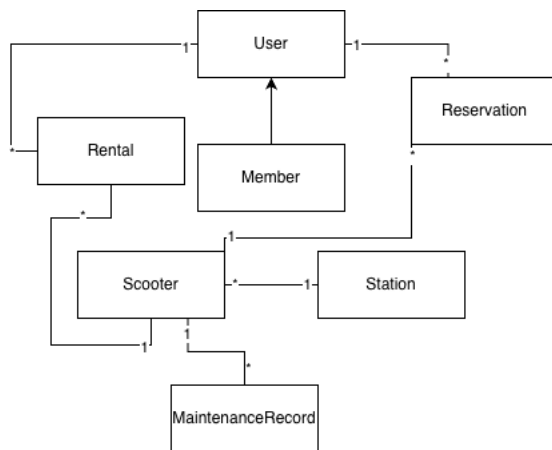
- user: User
- scooter: Scooter

MaintenanceRecord

- scooter: Scooter
- issue: string

The system follows object-oriented design principles such as encapsulation and inheritance.

4. Class Diagram



The class diagram shows how classes are connected and how data flows in the system.

The relationships show how objects are connected. These relationships help control how data moves between classes and make sure the system works correctly.

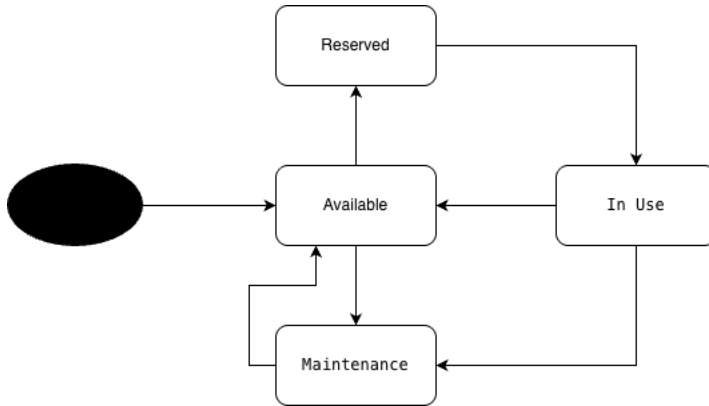
- One user can have many rentals (1..*)
- One scooter belongs to one station (1)
- One station has many scooters (1..*)
- One scooter can have many rentals and maintenance records

Assumption:

Each scooter can only be used by one user at a time.

5. State Diagrams

Scooter State Diagram



The scooter starts in available state. It can be reserved, then used. After use, it returns to available. If there is a problem, it moves to maintenance.

Transitions:

Available → Reserved (when user reserves)

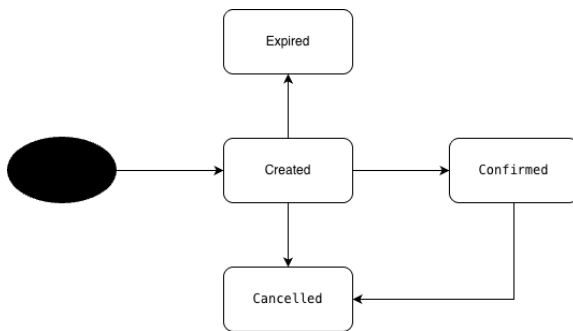
Reserved → In Use (when rental starts)

In Use → Available (after return)

In Use → Maintenance (if issue occurs)

Maintenance → Available (after repair)

Reservation State Diagram



A reservation starts as created. It can be confirmed, cancelled, or expired if not used.

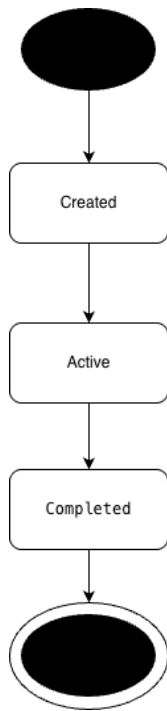
Transitions:

Created → Confirmed (user confirms)

Created → Cancelled (user cancels)

Created → Expired (time ends)

Rental State Diagram



A rental starts in created state. It becomes active when the user starts using the scooter. It becomes completed after the scooter is returned.

Transitions:

Created → Active (rental starts)

Active → Completed (rental ends)

6. Source Code

Main.py

```
from classes import *

# Create station
station1 = Station(1, "Main Station", 2)

# Create scooters
scooter1 = Scooter(101, 80, "available", "Station", "standard")
scooter2 = Scooter(102, 90, "available", "Station", "premium")

# Add scooters to station
station1.add_scooter(scooter1)
station1.add_scooter(scooter2)

# Try adding extra scooter (capacity check)
extra_scooter = Scooter(103, 70, "available", "Station")
station1.add_scooter(extra_scooter)

# Create user
user1 = Member(1, "Ali")

# Reserve scooter
reservation = user1.reserve_scooter(scooter1)

# Try renting reserved scooter (should fail)
user1.rent_scooter(scooter1)

# Set scooter back to available manually for testing
scooter1.status = "available"

# Rent scooter
rental = user1.rent_scooter(scooter1)

# End rental with duration
if rental:
    rental.end_rental(duration=2)

# Try renting same scooter again (should work now)
user1.rent_scooter(scooter1)

# Report maintenance
maintenance = MaintenanceRecord(scooter1, "Battery issue")

# Try renting scooter in maintenance (should fail)
user1.rent_scooter(scooter1)
```


Classes.py

```
# Classes for Scooter Rental System

class User:
    def __init__(self, user_id, name):
        self.user_id = user_id
        self.name = name

class Member(User):
    def __init__(self, user_id, name):
        super().__init__(user_id, name)
        self.rentals = []
        self.reservations = []

    def rent_scooter(self, scooter):
        # Prevent renting if scooter not available
        if scooter.status != "available":
            print("Scooter not available")
            return None

        scooter.status = "in use"
        rental = Rental(self, scooter)
        self.rentals.append(rental)
        print(self.name, "rented scooter", scooter.scooter_id)
        return rental

    def reserve_scooter(self, scooter):
        # Prevent reservation conflict
        if scooter.status != "available":
            print("Scooter cannot be reserved")
            return None

        reservation = Reservation(self, scooter)
        self.reservations.append(reservation)
        print(self.name, "reserved scooter", scooter.scooter_id)
        return reservation

class Scooter:
    def __init__(self, scooter_id, battery_level, status, location,
scooter_type="standard"):
        self.scooter_id = scooter_id
        self.battery_level = battery_level
        self.status = status
        self.location = location
        self.scooter_type = scooter_type # type affects cost
```

```

class Station:
    def __init__(self, station_id, name, capacity):
        self.station_id = station_id
        self.name = name
        self.capacity = capacity
        self.scooters = []

    def add_scooter(self, scooter):
        # Check station capacity
        if len(self.scooters) >= self.capacity:
            print("Station is full")
            return False

        self.scooters.append(scooter)
        print("Scooter added to station")
        return True

class Rental:
    def __init__(self, user, scooter):
        self.user = user
        self.scooter = scooter
        self.start_time = 0
        self.end_time = None
        self.cost = 0

    def end_rental(self, duration):
        # Calculate cost based on duration and scooter type
        rate = 5

        if self.scooter.scooter_type == "premium":
            rate = 10

        self.cost = duration * rate
        self.end_time = duration

        self.scooter.status = "available"

        print("Rental ended. Cost:", self.cost)

class Reservation:
    def __init__(self, user, scooter):
        self.user = user
        self.scooter = scooter

```

```
scooter.status = "reserved"

class MaintenanceRecord:
    def __init__(self, scooter, issue):
        self.scooter = scooter
        self.issue = issue

scooter.status = "maintenance"
```

Code Explanation

The system is built using object-oriented programming. Each class represents a part of the system such as user, scooter, station, rental, and reservation.

The **Member** class inherits from the **User** class. This shows inheritance. The Member can rent and reserve scooters.

The **Scooter** class stores scooter details like ID, status, and type. The type is used to calculate cost.

The **Station** class manages scooters and checks capacity so it does not exceed the limit.

The **Rental** class handles renting and calculates cost based on duration and scooter type. The system also checks scooter status to make sure one scooter is not used by multiple users at the same time.

The **Reservation** and **MaintenanceRecord** classes handle booking and faulty scooters.

Encapsulation is used by keeping data inside classes, and inheritance is used where Member extends User.

7. System Requirements Coverage

The system ensures:

- A scooter cannot be rented by multiple users at the same time
- Station capacity is controlled
- Rental cost is calculated based on duration and scooter type
- Scooters can be reserved and maintained

8. Conclusion

This project shows how object-oriented design can be used to build a real system. The system is simple and easy to understand.